Computer Graphics – Unit 1

Output Primitives · Line, Circle and Ellipse Algorithms · Polygon Filling

B.Tech CSE, 3rd Year · Classroom Notes

Note to students

Topics follow the list given by your faculty. Check the exact weightage and sub-topics against your university syllabus.

1. Basics: Pixels, Points and Lines

Definitions

Computer Graphics: creating, storing and displaying pictures using a computer.

Pixel: the smallest addressable picture element on a raster screen.

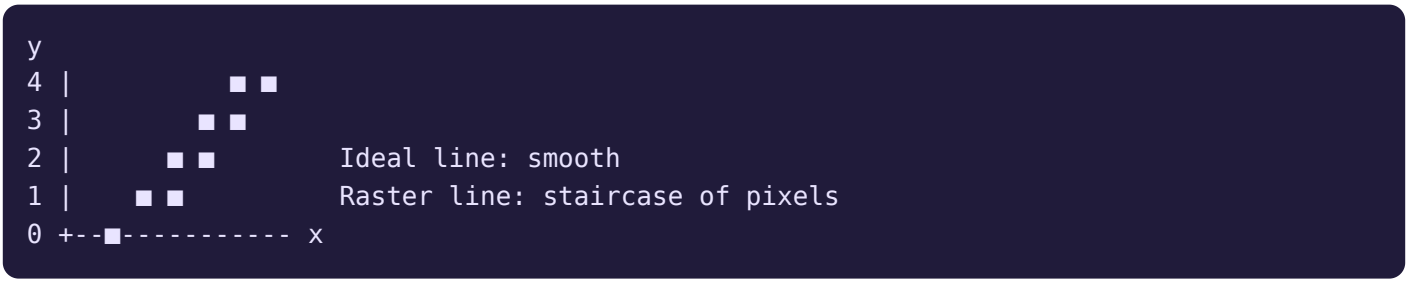
Raster display: the screen is a grid of pixels; a picture is stored in the *frame buffer*.

Output primitive: a basic shape (point, line, circle, ellipse, polygon, text) from which pictures are built.

Scan conversion (rasterization): converting a geometric shape into the set of pixels that best represent it.

Point: one pixel at (x, y). The origin (0,0) is usually the top-left corner of the screen, and y increases downwards.

Line: a screen has only whole-number pixel positions, so a real line must be approximated. This causes the staircase effect called aliasing (jaggies).



Real-world example

Zoom into a diagonal line in any photo or game screenshot; you will see small square steps. Those are pixels.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Slope form: $y = m \cdot x + c$, where $m = \Delta y / \Delta x = (y_2 - y_1)/(x_2 - x_1)$.

> **What makes a good line algorithm?**
> Straight, uniform brightness, starts and ends exactly at the endpoints, fast (integer maths preferred), works for all slopes.

2. DDA Line Drawing Algorithm

> **Definition**
> **DDA (Digital Differential Analyzer)** draws a line by taking equal small steps along the longer axis and calculating the other coordinate using the slope. It uses floating-point addition and rounding.

Algorithm

```
1. Input (x1,y1), (x2,y2)
2. dx = x2 - x1 ; dy = y2 - y1
3. steps = max(|dx|, |dy|)
4. Xinc = dx / steps ; Yinc = dy / steps
5. x = x1 ; y = y1 ; plot(round(x), round(y))
6. Repeat steps times:
       x = x + Xinc ; y = y + Yinc
       plot(round(x), round(y))
```

> **Worked example: line from (2,3) to (8,6)**
> dx = 6, dy = 3, steps = 6, Xinc = 1, Yinc = 0.5

k	x	y	Plot
0	2	3	(2,3)
1	3	3.5	(3,4)
2	4	4.0	(4,4)
3	5	4.5	(5,5)
4	6	5.0	(6,5)
5	7	5.5	(7,6)
6	8	6.0	(8,6)

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Rounding convention: 3.5 → 4 (round half up). State your convention in exams.

Advantages: simple, faster than using y = mx + c directly (no multiplication per step).
Disadvantages: floating-point work, rounding every step, and round-off error accumulates on long lines (drift).

3. Bresenham's Line Algorithm

Definition

Bresenham's algorithm draws a line using only integer addition, subtraction and shifts. A *decision parameter* chooses which of the two candidate pixels is nearer to the true line.

Idea (for 0 < m < 1)

```
At each step x increases by 1. y either stays or goes up by 1.

        (xk+1, yk+1)  ○  upper candidate
   true line  ----------*---
        (xk+1, yk)    ○  lower candidate
   Pick the pixel closer to the true line.
```

Algorithm (0 < m < 1)

```
1. dx = x2 - x1 ; dy = y2 - y1
2. p0 = 2*dy - dx
3. Plot (x1, y1)
4. For each x from x1 to x2-1:
     if p < 0 : next pixel (x+1, y) ; p = p + 2*dy
     else     : next pixel (x+1, y+1) ; p = p + 2*dy - 2*dx
```

Worked example: (20,10) to (30,18)

dx = 10, dy = 8, 2dy = 16, 2dy − 2dx = −4, p0 = 16 − 10 = 6

k	pk	Next pixel
0	6	(21,11)
1	2	(22,12)
2	−2	(23,12)
3	14	(24,13)

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

4	10	(25,14)
5	6	(26,15)
6	2	(27,16)
7	−2	(28,16)
8	14	(29,17)
9	10	(30,18)

Other slopes

For m > 1 swap the roles of x and y. For negative slopes, decrement the coordinate that decreases. Always handle all cases in a full program.

DDA vs Bresenham

Point	DDA	Bresenham
Arithmetic	Floating point	Integer only
Rounding	Needed every step	Not needed
Accuracy	Error can accumulate	More accurate
Speed	Slower	Faster
Hardware	Less suitable	Easy to implement
Complexity	Simple to understand	Slightly harder

Real-world use

Bresenham-style integer line drawing is used in plotters, embedded displays, games and rasterizer hardware.

4. Mid-Point Circle Algorithm

Definition

A **circle** is the set of points at a fixed distance *r* from a centre. Centred at the origin: $x^2 + y^2 = r^2$. The **mid-point circle algorithm** uses integer arithmetic and a decision parameter based on the midpoint between two candidate pixels.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Key idea: 8-way symmetry

Compute only one octant (from (0,r) to x = y); the other seven come by symmetry. For each (x,y) plot: (±x,±y) and (±y,±x), shifted by the centre (xc,yc).

```
      y
   \ 2 | 1 /
  3 \  |  / 0       Compute octant 1 only
 ----- + ----- x    then reflect to the rest
  4 /  |  \ 7
   / 5 | 6 \
```

Decision parameter

Circle function: $f(x,y) = x^2 + y^2 - r^2$. Negative inside, zero on, positive outside. Start at (0, r). $p_0 = 1 - r$ (integer form).

```
x = 0 ; y = r ; p = 1 - r
while x < y:
    x = x + 1
    if p < 0 : p = p + 2x + 1
    else     : y = y - 1 ; p = p + 2x + 1 - 2y
    plot 8 symmetric points
```

Worked example: r = 10, centre (0,0)

$p_0 = 1 - 10 = -9$

k	pk	(x,y)
0	−9	(1,10)
1	−6	(2,10)
2	−1	(3,10)
3	6	(4,9)
4	−3	(5,9)
5	8	(6,8)
6	5	(7,7)

Stop when x ≥ y.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

5. Mid-Point Ellipse Algorithm

> **Definition**
> An **ellipse** centred at the origin with semi-axes rx and ry: `x²/rx² + y²/ry² = 1`. Ellipse function: f(x,y) = $ry^2x^2 + rx^2y^2 - rx^2ry^2$.

Ellipse has only 4-way symmetry, so one quadrant is computed, in **two regions**.

```
        y
        |   Region 1: slope |m| < 1 (step in x)
     ___|___
    /   |   \   Region 2: slope |m| > 1 (step in y)
---(----+----)--- x
    \___|___/   Boundary: where 2·ry²·x ≥ 2·rx²·y
```

	Region 1	Region 2
Start	(0, ry)	Last point of region 1
Step	x always +1	y always −1
p initial	$ry^2 - rx^2 \cdot ry + \frac{1}{4} \cdot rx^2$	$ry^2(x+\frac{1}{2})^2 + rx^2(y-1)^2 - rx^2ry^2$
p < 0	y same, p += $2ry^2x + ry^2$ (using the new x)	x +1, p += $2ry^2x - 2rx^2y + rx^2$ (new x,y)
p ≥ 0	y−1, p += $2ry^2x - 2rx^2y + ry^2$ (new x,y)	x same, p += $rx^2 - 2rx^2y$ (new y)
Stop	$2ry^2x \geq 2rx^2y$	y = 0

> **Remember**
> If rx = ry the ellipse becomes a circle. Plot 4 points per step: (±x, ±y) + centre.

Circle vs Ellipse algorithm

Point	Circle	Ellipse
Symmetry	8-way	4-way
Regions	1 octant	2 regions in 1 quadrant
Initial p	1 − r	Region-dependent (may involve fractions)

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

6. Polygon Filling

Definitions

Polygon: closed figure made of straight edges.

Polygon filling: colouring all interior pixels.

Convex polygon: any line joining two interior points stays inside. **Concave:** not so.

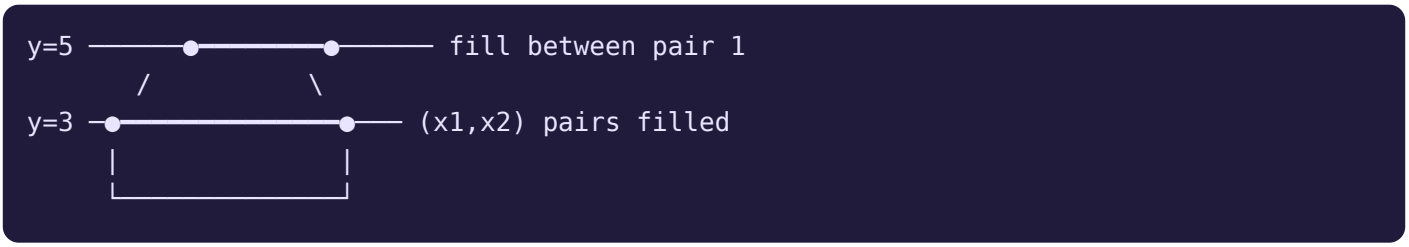
4-connected: pixel has neighbours left, right, up, down. **8-connected:** also the four diagonals.

Real-world example

The "paint bucket" tool in MS Paint is a flood-fill. Games and CAD fill shapes with scan-line methods.

6.1 Scan-Line Algorithm

Idea: move a horizontal line from bottom to top. On each scan line, find where it crosses the polygon edges, sort the intersections by x, and fill between pairs.



Steps

1. Find ymin and ymax of the polygon.
2. For each scan line y: find intersections with edges.
3. Sort intersections by x.
4. Fill pixels between pairs (1st–2nd, 3rd–4th, ...) using the odd-even (parity) rule.
5. Move to next y.

Special cases

Horizontal edges: ignore them.

Vertex on scan line: if the two edges at the vertex lie on opposite sides of the scan line, count it once; if both on the same side (local min/max), count twice or zero. A common trick: shorten the upper end of each edge by one scan line.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Edge coherence: x of intersection on next line = x + 1/m. Active Edge List and Edge Table speed this up.

6.2 Boundary-Fill Algorithm

Definition

Start from an interior seed pixel and spread outward until the **boundary colour** is reached. Works when the boundary has one single colour.

```
boundaryFill(x, y, fill, boundary):
    c = getPixel(x, y)
    if c != boundary and c != fill:
        setPixel(x, y, fill)
        boundaryFill(x+1, y, ...)   boundaryFill(x-1, y, ...)
        boundaryFill(x, y+1, ...)   boundaryFill(x, y-1, ...)
        // add 4 diagonals for 8-connected
```

6.3 Flood-Fill Algorithm

Definition

Replace a connected region of one **interior (old) colour** with a new colour, starting at a seed point. Works when the region is not bounded by a single colour.

```
floodFill(x, y, newColor, oldColor):
    if getPixel(x, y) == oldColor:
        setPixel(x, y, newColor)
        floodFill(x+1,y) floodFill(x-1,y)
        floodFill(x,y+1) floodFill(x,y-1)
```

Warning

Recursive versions can cause **stack overflow** on big regions. Use an explicit stack/queue or span (scan-line) fill.

4-connected vs 8-connected fill

```
4-connected leaks?  No: diagonal gaps stop it.
  ■ ■ ■          8-connected may leak
  ■ . ■  ← seed  through a diagonal gap in
  ■ ■ .    gap   a thin boundary.
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Comparison of filling methods

Point	Scan-line	Boundary-fill	Flood-fill
Needs seed?	No	Yes	Yes
Stops at	Edge intersections	Boundary colour	Change from old colour
Best for	Vector polygons	Single-colour border	Multi-colour borders, raster images
Method	Geometric	Pixel/recursive	Pixel/recursive
Speed	Fast	Slower	Slower
Memory	Low (edge lists)	High if recursive	High if recursive

7. Common Student Mistakes

- Forgetting to plot the starting point in DDA/Bresenham tables.
- Using Bresenham's formula $p_0 = 2dy - dx$ for slopes > 1 without swapping x and y.
- Mixing up the two update formulas for $p < 0$ and $p \geq 0$.
- Rounding inconsistently in DDA (3.5 → 3 in one row, 4 in another).
- Stopping the circle algorithm at the wrong point (it is $x < y$, not $x < r$).
- Forgetting to add the centre (xc, yc) when the circle is not at the origin.
- Calling 4-point symmetry for a circle or 8-point for an ellipse.
- Mixing up boundary-fill (stops at boundary colour) with flood-fill (replaces old colour).
- Not handling vertices and horizontal edges in the scan-line method.
- Saying DDA is "inaccurate". It is *less accurate for long lines* because of accumulated rounding.

8. Interview Questions

1. What is scan conversion?
2. Why is Bresenham's algorithm faster than DDA?
3. What is aliasing? How can it be reduced? (Hint: anti-aliasing.)
4. Why do we compute only one octant for a circle?

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

5. What is the role of the decision parameter?
6. Difference between 4-connected and 8-connected regions?
7. Why can recursive flood-fill crash? What is the alternative?
8. When would you prefer scan-line fill over flood-fill?
9. Can Bresenham's method be used for circles? (Yes; the circle algorithms are of the same integer-incremental family.)
10. How does the paint-bucket tool work?

9. University Examination Questions

1. Define output primitive and scan conversion. (2–3 marks)
2. List the advantages and disadvantages of the DDA algorithm.
3. Rasterize the line from (1,1) to (8,5) using DDA.
4. Rasterize the line from (2,3) to (10,9) using Bresenham's algorithm.
5. Write the mid-point circle algorithm. Plot a circle of radius 8 with centre (0,0).
6. Explain the mid-point ellipse algorithm with its two regions.
7. Distinguish between boundary-fill and flood-fill.
8. Explain the scan-line polygon fill algorithm with a neat diagram.
9. Write short notes on: (a) 8-way symmetry (b) 4- and 8-connected pixels (c) aliasing.

10. Five-Minute Classroom Activity: "Be the Pixel"

Setup (pairs, 5 minutes)

1. **(1 min)** Draw a 10×10 grid. Partner A gives two points, e.g. (1,1) and (9,5).
2. **(2 min)** Partner B finds the pixels using Bresenham, Partner A uses DDA.
3. **(1 min)** Compare grids. Are they the same? Where do they differ?
4. **(1 min)** Pick a seed inside a small shape and show the order in which flood-fill visits pixels.

Expected outcome: students see that both methods give almost the same line, but Bresenham needs no floating point.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

11. Short Recap

- **Pixel** is the smallest unit; **scan conversion** turns shapes into pixels.
- **DDA:** floating point, steps = max(|dx|,|dy|).
- **Bresenham:** integer only, $p_0 = 2dy - dx$ ($m < 1$).
- **Mid-point circle:** $p_0 = 1 - r$, 8-way symmetry, stop at $x \geq y$.
- **Mid-point ellipse:** 4-way symmetry, two regions.
- **Scan-line:** intersections + sort + pair-fill (odd-even rule).
- **Boundary-fill:** stops at boundary colour. **Flood-fill:** replaces old colour.

12. Five Long-Answer Questions

1. Explain the DDA algorithm with a flowchart or pseudo-code. Rasterize the line from (5,5) to (13,9), and list its limitations.
2. Derive the decision parameter of Bresenham's line algorithm for $0 < m < 1$. Rasterize (12,9) to (22,15).
3. Explain the mid-point circle algorithm with 8-way symmetry. Find the points in the first octant for $r = 12$.
4. Describe the mid-point ellipse algorithm, including the region boundary condition and both decision parameters.
5. Compare the scan-line, boundary-fill and flood-fill algorithms. Explain one in detail with pseudo-code and diagrams, and discuss 4- vs 8-connected filling.

Tip for converting to PDF

Use your browser's Print → Save as PDF. Colours are preserved and tables/boxes avoid breaking across pages.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF